

Laboratory 5: DHT Sensor

Objective:

1. Install and manage external libraries in Arduino IDE
2. Read sensor data from a DHT temperature and humidity sensor
3. Handle sensor read errors using the `isnan()` error-checking function
4. Display formatted sensor data on the M5Stack LCD screen
5. Use button inputs to control how data is displayed
6. Combine multiple inputs (sensor + buttons) to create an interactive thermometer device

Key Concepts

What is a Library?

A library is a pre-written package of code that handles complex tasks for us. Just like a library of books contains knowledge you can use without writing your own books, an Arduino library contains functions you can use without writing all the low-level code yourself.

Real-world analogy: Imagine you want to make a phone call. Instead of building a phone from scratch, you buy one from a store (the library) and just press buttons to use it. The library does all the complex work behind the scenes.

In this lab, the DHT Sensor Library handles all the low-level communication with the sensor. We simply call its functions to get temperature and humidity values.

DHT Sensor

The DHT sensor is a small electronic device that measures two things about the air around it: - Temperature (in degrees Celsius or Fahrenheit) - Humidity (how much moisture is in the air, as a percentage). Think of it like a tiny weather station. It has three pins:

- **VCC (+):** Power (connects to 3.3V)
- **DATA (out):** Sends sensor readings as digital signals (connects to GPIO pin 21)
- **GND (-):** Ground (connects to GND)

DHT11/DHT22 pins and connection to M5Stack are shown in illustration 5.1. In the lab, we used DHT22 with three pins.

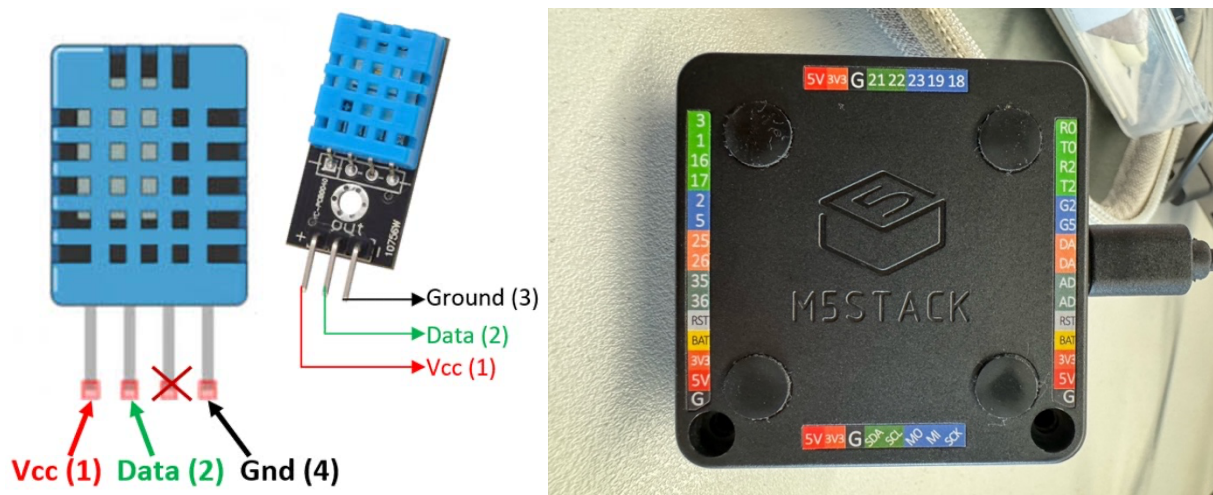


Illustration 5.1 DHT11/DHT22 Pins and M5Stack

Data Types: **int** vs **float**

In previous labs, we used integers (**int**), which are whole numbers: 1, 2, 3, etc.

Sensors often return floating-point numbers (**float**), which can have decimal places: 23.47, 45.82, etc. This is important because temperature and humidity are not always whole numbers.

```
int count = 5;           // Whole number
float temperature = 23.47; // Number with decimals
```

Sensor Readings Can Fail — **isnan()**

Sometimes, the sensor gives a bad reading. This can happen due to: - Loose wiring - Sensor not yet ready (needs a moment after power-on) - Electrical interference

When a sensor read fails, the library returns a special value called **NaN** (Not a Number).

The function **isnan()** checks if a value is **NaN**:

```
if (isnan(temperature)) {
    Serial.println("Sensor read failed!");
    // Don't trust this value
} else {
    Serial.println("Sensor reading is valid");
    // Safe to use this value
}
```

This is very important—if we don't check for **NaN**, we might display garbage data to the user.

Why Wait Between Sensor Readings?

The DHT sensor is not very fast. It needs about 2 seconds to collect and process a new measurement. If we read it more frequently than that, we'll just get the same old value, or sometimes get read errors.

Real-world analogy: If you ask a person the same question 10 times in 1 second, they can't think of a new answer—they'll repeat the old one or get confused.

```
delay(2000); // Wait 2 seconds before reading sensor again
float temp = dht.readTemperature(); // Now safe to read
```

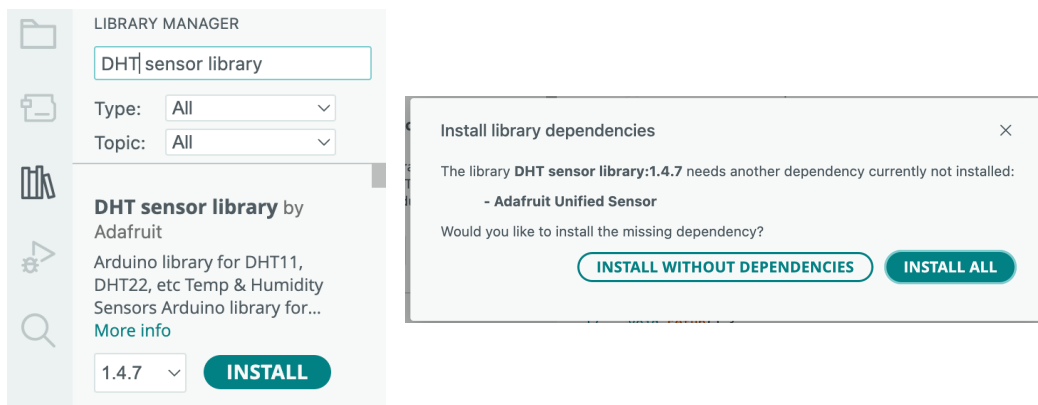
Lab Exercises

Task 0.1: Install DHT Library

Objective: Set up the Arduino environment to use the DHT sensor library.

1. Open **Arduino IDE**. **DO NOT** install any update, click **LATER** if you see update window pop-up.
2. Go to **Tools -> Manage Libraries** (opens the Library Management window)
3. In the search box, type: **DHT Sensor Library**
4. Find the library by Adafruit (usually at the top)
5. Click **"INSTALL"** → **"INSTALL ALL"** to install all dependencies.
6. Wait for the download to complete. You should see "Installation complete" message.
7. Close the Library Manager.

Note: You only need to do this once. All future sketches can use the DHT library.



Task 0.2: Connecting the DHT sensor to M5Stack

Objective: Wire the DHT sensor correctly to prevent sensor damage.

1. Identify DHT22 pins.

Looking at DHT22 from the front (with labels facing you):

- Pin 1 (left, '+'): VCC (+3.3V)
- Pin 2 (middle, out): DATA (Signal)
- Pin 3 (right, '-'): GND (Ground)

2. Disconnect M5Stack from USB (if connected)
3. Connect GND first (safest)

DHT22 GND Pin 3(-) --[black wire]--> M5Stack G

4. Connect VCC

DHT22 Pin 1(+) --[red wire]--> M5Stack 3V3 (not 5V)

5. Connect data pin

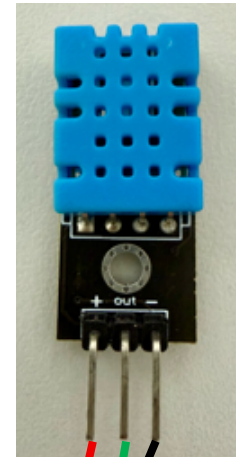
DHT22 Pin 2(out) --[other color wire]-->
M5Stack Pin 21

6. Verify before connecting M5Stack to USB

- ☐ GND (-) is connected to G
- ☐ VCC (+) is connected to 3V3
- ☐ Out is connected to Pin 21
- ☐ All connections are firm

7. USB connection steps

- 7.1. Disconnect USB cable from computer
- 7.2. Plug USB into M5Stack (not into computer yet)
- 7.3. Plug USB into Computer (make sure that connection is "connected")



Task 1: First Sensor Reading (lab_5_1)

Objective: Read DHT sensor data correctly

- 1.1 Copy the given code into Arduino IDE and save as **lab_5_1**.

```
#include <M5Unified.h>
#include "DHT.h"

// ===== SENSOR CONFIGURATION =====
// DHTPIN: The GPIO pin that the sensor's DATA line is connected to
// DHTTYPE: The sensor model (DHT22 is more accurate; DHT11 is cheaper)
#define DHTPIN 21
#define DHTTYPE DHT22

// ===== GLOBAL VARIABLES =====
// Create a DHT object to communicate with the sensor
```

```
DHT dht(DHTPIN, DHTTYPE);

// Track how many successful readings we've had
int reading_count = 0;

void setup() {
  // Initialize M5Stack display, buttons, and speaker
  M5.begin();

  // Set LCD display preferences
  M5.Display.setTextSize(3);          // Medium text size
  M5.Display.println("Lab 5 - DHT Sensor");

  // Initialize serial communication for debugging
  Serial.begin(115200);
  Serial.println("Lab 5 - DHT Sensor");

  // Initialize the DHT sensor
  dht.begin();
  Serial.println("DHT sensor initialized. Waiting for first reading...");
}

void loop() {
  // ===== STEP 1: WAIT FOR SENSOR STABILITY =====
  // Don't read too frequently—the sensor needs time to settle
  delay(2000);

  // ===== STEP 2: READ SENSOR VALUES =====
  // readHumidity() returns humidity as a floating-point percentage (e.g.,
  45.5)
  float humid = dht.readHumidity();

  // readTemperature() returns temperature in Celsius by default
  float tempC = dht.readTemperature();

  // readTemperature(true) returns temperature in Fahrenheit
  float tempF = dht.readTemperature(true);

  // ===== STEP 3: ERROR CHECKING =====
  // If the sensor read failed, it returns NaN (Not a Number)
  // isnan() checks if a value is NaN
  // We should NEVER display NaN to the user—it's meaningless
  if (isnan(humid) || isnan(tempC) || isnan(tempF)) {
    Serial.println("ERROR: Sensor read failed! Skipping this reading.");
    return; // Skip the rest of this loop iteration
  }

  // ===== STEP 4: PROCESS DATA =====
  // Count how many successful readings we've had
  reading_count++;

  // ===== STEP 5: DISPLAY ON SERIAL MONITOR =====
  // Serial.printf() is like printf() in C—it formats text with placeholders
  // %03d = integer with leading zeros (e.g., 001, 042, 100)
  // %.2f = floating-point number with 2 decimal places (e.g., 23.47)
  // %% = a single percent sign (%)
  Serial.printf("%03d | ", reading_count);
```

```

Serial.printf("Humidity: %.2f%% | ", humid);
Serial.printf("Temp: %.2fC / %.2fF\n", tempC, tempF);

// ===== STEP 6: DISPLAY ON LCD =====
update_LCD(reading_count, humid, tempC, tempF);
}

void update_LCD(int count, float humidity, float celsius, float fahrenheit) {
    // Clear the LCD before displaying new data
    M5.Display.clear();
    M5.Display.setCursor(0, 0);
    // Display the reading number
    M5.Display.print("Reading #");
    M5.Display.println(count);
    // Display humidity with formatting
    M5.Display.printf("Humidity: %.2f%%\n", humidity);
    // Display temperature in Celsius
    M5.Display.printf("Temp: %.2fC\n", celsius);
}

```

1.2 Upload the sketch

1.3 **Observe:**

- Do you see message in the Serial Monitor?
- Does the LCD on M5Stack show reading?
- Does the reading count go up every 2 seconds?

If nothing appears: Check the wiring (is the out pin really connected to pin 21?)

If you see “Sensor read failed”: Check the wiring if it is loose. If the sensor is hot, the sensor might be damage. Request for a new sensor and repeat **Task 0.2** carefully.

1.4 **Task 1 complete when:** Serial monitor and LCD show reading count, humidity and temperature every 2 seconds.

1.5 Answer question Q1.

Task 2. Optimize LCD Display (lab_5_2)

Objective: Only update the LCD when the humidity or temperature actually changes (not every 2 seconds).

Why? Constantly redrawing the LCD wastes energy. If the reading hasn’t changed significantly, we should skip the LCD update.

2.1 Replace the `loop()` from **Task 1** with the following code, then save as **lab_5_2**.

```

void loop() {

    // TODO 2.2: Add code from STEP 1 to STEP 5 (same as Task 1)
    // ===== STEP 1: WAIT FOR SENSOR STABILITY =====

    // ===== STEP 2: READ SENSOR VALUES =====

```

```
// ===== STEP 3: ERROR CHECKING =====

// ===== STEP 4: PROCESS DATA =====

// ===== STEP 5: DISPLAY ON SERIAL MONITOR =====

// UPDATE STEP 6: DISPLAY ON LCD ONLY WHEN NEEDED =====
bool humid_changed;
bool temp_changed;
// TODO 2.3: Check if humidity or temperature changed significantly
// the changed is significant when the different between
// current reading and previous reading reaches the threshold:
// |current_reading - previous_reading| >= threshold
// You need to update humid_changed and temp_changed before if statement
humid_changed = ?
temp_changed = ?

// humid_changed, temp_change must be initialized
if (humid_changed || temp_changed) {
    // log to Serial show that this reading_count has some change
    // Update LCD here: LCD is updated only when needed
}

// TODO 2.4: Update the previous readings
}
```

TODO 2.1: Add these global variables at the top (after `int reading_count = 0;`):

```
float prev_humid = -100.0; // Store the previous humidity reading
float prev_tempC = -100.0; // Store the previous temperature reading
float HUMID_THRESHOLD = 0.2; // threshold for humidity changed
float TEMP_THRESHOLD = 0.1; // threshold for temperature changed
```

TODO 2.2: Add **STEP 1** to **STEP 5** for DHT Sensor reading.

TODO 2.3: Check if humidity or temperature changed significantly. The changed is significant when the different between current reading and previous reading reaches the threshold:

```
|current reading - previous reading| >= threshold
```

TODO 2.4: Update the previous readings, since the current readings will be the previous readings of next readings.

Answer question Q2.

Task 3. Toggle LCD Display ON/OFF (lab_5_3)

Objective: Use Button A to turn the LCD display ON and OFF. When OFF, the LCD is dark. When ON, it shows the sensor data.

TODO 3.1: Add a new global variable to store LCD state.

```
bool display_enabled = true; // Track whether LCD is ON or OFF
```

TODO 3.2: Add Button A handler code. Follow the step given in pseudo code below:

```
Check for Button A PRESSED:
  Log event to Serial Monitor
  Toggle display_enabled (similar to toggle led_state)

Check if the display is enabled?
When enabled:
  Log "Display ON" to Serial Monitor
  Set LED brightness to full
When not enabled:
  Log "Display OFF" to Serial Monitor
  Set LED brightness to zero (dark)
```

TODO 3.3: Update the LCD display logic. The `update_LCD()` should be called only when the display is enabled.

Hint: You can turn LCD OFF or ON by set the brightness of the LCD using

```
M5.Display.setBrightness(0) or M5.Display.setBrightness(255)
```

Test it:

- Upload the sketch. When start, the program must have the same behavior as in Task 2.
- Press Button A. Serial Monitor shows "Button A is pressed" message and display state.
- Press Button A again. The LCD should show data normally. Serial Monitor shows "Button A is pressed" message and display state.

Answer question Q3.

Task 4: Interactive Thermometer (lab_5_4)

Objective: Combine all previous concepts to build a practical thermometer where buttons control what is displayed.

Requirements:

- **Button A:** Toggles the LCD display ON/OFF.
- **Button B:** Shows temperature in **Fahrenheit** when LCD is displayed.
- **Button C:** Shows temperature in **Celsius** when LCD is displayed.
- Log all button events to Serial Monitor. (Show message "Button A was pressed", "Button B was pressed", or "Button C was pressed" on Serial Monitor according to which button is pressed.)

Modify the sketch from Task 3, then save as **lab_5_4**. After finish, **answer question Q4**.

Hint: To manage the display **Fahrenheit/Celsius**, use a global variable to store the current unit (e.g., Celsius). For example, `bool showCelsius = true;`. When button **B/C** is pressed, update that variable. In function `update_LCD()`, use a conditional statement to check this variable and display the appropriate value.

**Lab 5: DHT Sensor**

Section: _____ Date: _____

Members

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Task		Graded by
1	Reading DHT Sensor data Serial Monitor and LCD shows info every 2 seconds	
2	Optimize LCD display Serial Monitor shows every reading LCD update when changes reach threshold	
3	Turn LCD ON/OFF Button A toggles LCD ON/OFF Serial Monitor shows every reading + Button events	
4	Interactive Thermometer (B-Fahrenheit, C-Celsius) Button A toggle LCD ON/OFF Button B shows Fahrenheit Button C shows Celsius Serial Monitor shows every reading + Button events	

5. Answer question

[Answer after completing Task 1]

Q1. When you first uploaded the code, did the sensor start reading immediately, or did you have to wait? What does the serial monitor message “Waiting for first reading...” tell you about how long sensors need to initialize?

[Answer after completing Task 2]

Q2. In **Task 2**, you added a check to skip LCD updates when values don't change much. What happens to the Serial Monitor output—does it also skip updates, or does it continue to print every 2 seconds? Why would we want different update rates for Serial Monitor vs. LCD?

[Answer after completing Task 3]

Q3. When you press Button A, what exactly does `M5.Display.setBrightness(0)` do? Can you still see any text on the screen when brightness is 0? What does this tell you about how LCD displays work?

[Answer after completing Task 4]

Q4. In this lab, you used different thresholds for LCD update: when temperature changes by at least 0.1°C, or humidity changes by at least 0.2%. If you want the LCD to update even less frequently (to save power), how should you adjust these thresholds? Why?

6. Reflection about this lab. See assignment in **MycourseVille**